

Sistemas Distribuidos y Paralelos

Ingeniería en Computación



MultiGPUs y modelos híbridos

Universidad Nacional de La Plata



Facultad de Informática





1

Múltiples GPUs y modelos híbridos

2

Múltiples GPUs

3

Modelos híbridos

CUDA + OpenMP. CUDA + Pthreads. CUDA + MPI. CUDA + OpenMP + MPI. CUDA + Pthreads + MPI. Modularidad. Distribución de carga.



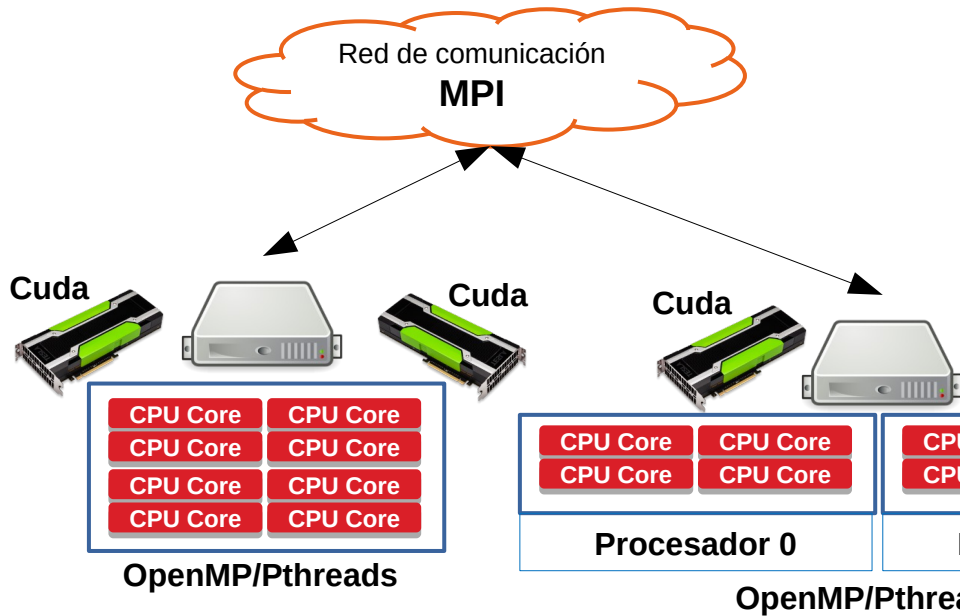
1

Múltiples GPUs y modelos híbridos

Múltiples GPUs y modelos híbridos

4

- Actualmente contamos con cluster formados por nodos multicore con una o varias GPUS.
 - Podemos considerar 3 niveles de hardware: Cada nivel requiere una herramienta específica para programar la arquitectura subyacente.



Hardware	Herramienta
Coprocesadores/Manycores GPU/MultiGPU	Cuda, OpenCL, Sycl etc.
Multicores	Modelo de memoria compartida: • OpenMP • Pthreads Modelo de memoria distribuida: • MPI
Cluster/Multicluster	Modelo de memoria distribuida: • MPI

Múltiples GPUs y modelos híbridos

5

- Programar sobre estos modelos híbridos conlleva complejidad que se presenta al integrar el hardware y el software.
- Algunas herramientas que facilitan la programación:
 - OpenACC
 - Intel OpenAPI
- Pero si queremos tener control absoluto debemos programar sin abstraernos.
- Para un correcto desarrollo se deben tener en cuenta distintos aspectos:
 - Compilación sobre arquitecturas diferentes
 - Compilación en módulos separados para cada arquitectura
 - Enlazar los módulos de cada arquitectura
 - Integrar diversas opciones de compilación



2

Múltiples GPUs

Múltiples GPUs

7

- En una máquina con más de una GPU debemos especificar sobre que GPU se están realizando las acciones. Para esto, se utiliza la sentencia:

```
cudaSetDevice(int id_device);
```

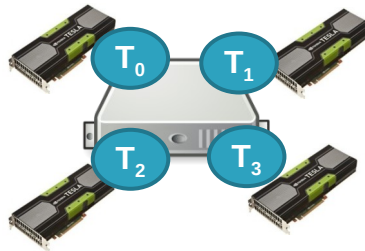
- `id_device`: número entero entre 0 y el número de GPUs - 1 instaladas.

```
...  
cudaSetDevice(0); //Todo lo que se haga de acá en más: sobre GPU0  
cudaMalloc(...);  
cudaMemcpy(...);  
Kernel_0<<<...>>>();  
...  
cudaSetDevice(1); //Todo lo que se haga de acá en más: sobre GPU1  
cudaMalloc(...);  
cudaMemcpy(...);  
Kernel_1<<<...>>>();  
...
```

Múltiples GPUs

8

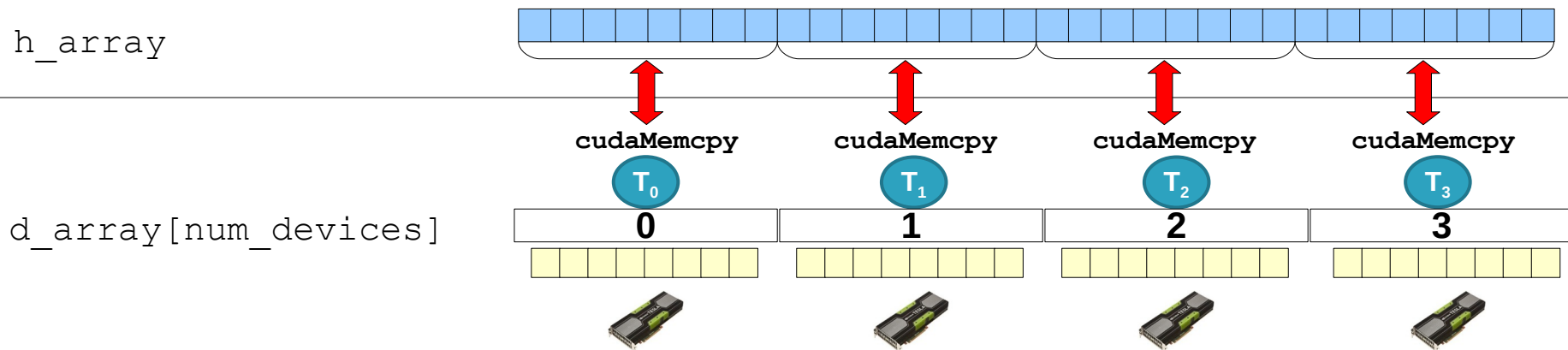
- Para realizar el cómputo correcto de un conjunto de datos utilizando varias GPUs, se suele crear un hilo por cada GPU disponible.
- El identificador de cada hilo debería coincidir con el `id_device`.
- Cada hilo se encarga de gestionar todo lo relacionado con su propia GPU.



Múltiples GPUs

9

- Ejemplo: Se quiere realizar cómputo sobre N elementos almacenados en un array.
 - El cómputo de una porción del array es independiente del resto.
 - Suponemos `num_devices` GPUs iguales, los datos se distribuyen proporcionalmente entre ellas ($\text{porción} = N / \text{num_devices}$)
 - Las porciones del array en GPU conviene gestionarlas mediante un array indexado por `id_device` (o identificador del hilo).
 - Cada hilo debe calcular la posición del array a copiar en su porción en GPU.



Modelos híbridos

CUDA + OpenMP

CUDA + Pthreads

CUDA + MPI

CUDA + OpenMP + MPI

CUDA + Pthreads + MPI

Modularidad

Distribución de carga

Modelos híbridos

CUDA + OpenMP

CUDA + Pthreads

CUDA + MPI

CUDA + OpenMP + MPI

CUDA + Pthreads + MPI

Modularidad

Distribución de carga

CUDA + OpenMP

12

- Se asigna un hilo para gestión de cada GPU y el resto de los hilos a cómputo:



- El compilador CUDA está basado en GCC. Por lo tanto, pueden utilizarse las opciones de GCC. Un programa CUDA+OpenMP se compila:

```
nvcc -Xcompiler -fopenmp programa.cu
```


Modelos híbridos

CUDA + OpenMP

CUDA + Pthreads

CUDA + MPI

CUDA + OpenMP + MPI

CUDA + Pthreads + MPI

Modularidad

Distribución de carga

CUDA + Pthreads

14

- Con Pthreads se sigue la misma estrategia que OpenMP.



- Un programa CUDA+Pthreads se compila:

```
nvcc -pthread programa.cu
```

Modelos híbridos

CUDA + OpenMP

CUDA + Pthreads

CUDA + MPI

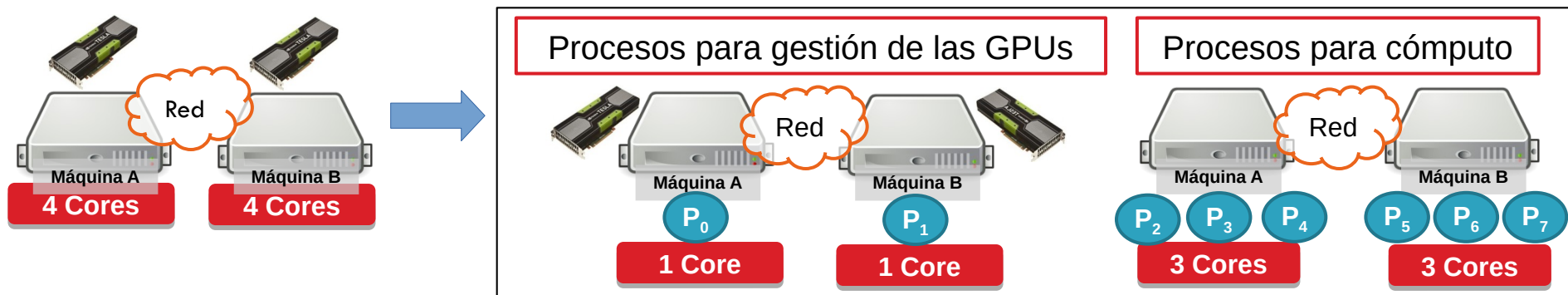
CUDA + OpenMP + MPI

CUDA + Pthreads + MPI

Modularidad

Distribución de carga

- Siguiendo la idea de memoria compartida, se asigna un proceso de gestión por GPU y el resto a cómputo:



- Puede compilarse con `nvcc` o `mpicc`. La forma más simple es hacerlo con `nvcc`. Un programa CUDA+MPI se compila:

```
nvcc -I$MPI_PATH/include -L$MPI_PATH/lib -lmpi programa.cu
```

- Luego, se ejecuta con `mpirun`

Modelos híbridos

CUDA + OpenMP

CUDA + Pthreads

CUDA + MPI

CUDA + OpenMP + MPI

CUDA + Pthreads + MPI

Modularidad

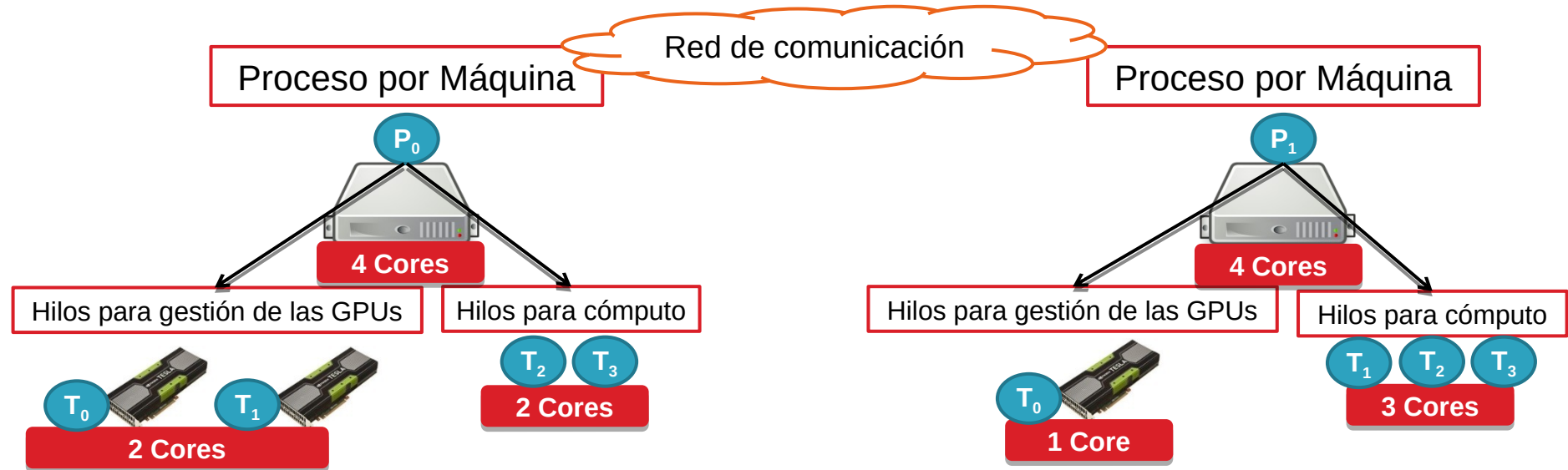
Distribución de carga

CUDA + OpenMP + MPI

CUDA + Pthreads + MPI

18

- Se asigna un proceso por máquina del cluster y este proceso crea los hilos.
- Luego se sigue la estrategia de memoria compartida (se crea y asigna un hilo de gestión por cada GPU y el resto a cómputo):



CUDA + OpenMP + MPI

CUDA + Pthreads + MPI

19

- La forma más sencilla de compilar es utilizando `nvcc`:

- Para compilar Cuda+OpenMP+MPI:

```
nvcc -Xcompiler -fopenmp -I$MPI_PATH/include -L$MPI_PATH/lib -lmpi programa.cu
```

- Para compilar Cuda+Pthreads+MPI:

```
nvcc -I$MPI_PATH/include -L$MPI_PATH/lib -lmpi programa.cu
```

- Luego, se ejecuta con `mpirun`

Modelos híbridos

CUDA + OpenMP

CUDA + Pthreads

CUDA + MPI

CUDA + OpenMP + MPI

CUDA + Pthreads + MPI

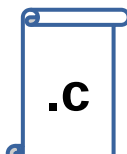
Modularidad

Distribución de carga

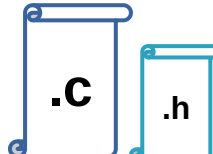
Modularidad

21

- Lo anterior asume un único archivo `.cu` que integra todas las herramientas.
- Para lograr mayor **modularidad**, es conveniente ubicar el código de cada herramienta en archivos separados.

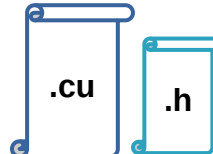


`mpi_source.c`



`omp_source.c`
`omp_source.h`

`pthread_source.c`
`pthread_source.h`



`cuda_source.cu`
`cuda_source.h`

Modularidad

CUDA + OpenMP

22

- Compilación con un **único compilador**:

```
nvcc -Xcompiler -fopenmp omp_source.c cuda_source.cu
```

- **Compilación separada**:

- **Compilar** cada herramienta con su compilador:

- **OpenMP**: `gcc -fopenmp -c omp_source.c -o omp_source.o`

- **CUDA**: `nvcc -c cuda_source.cu -o cuda_source.o`

- **Linkear** los archivos de salida (.o) con un compilador:

```
gcc cuda_source.o omp_source.o -L$CUDA_PATH/lib64 -lgomp -lcudart
```

Modularidad

CUDA + Pthreads

23

- Compilación con un **único compilador**:

```
nvcc pthreads_source.c cuda_source.cu
```

- **Compilación separada**:

- **Compilar** cada herramienta con su compilador:

- **Pthreads**: `gcc -pthread -c pthreads_source.c -o pthreads_source.o`
- **CUDA**: `nvcc -c cuda_source.cu -o cuda_source.o`

- **Linkear** los archivos de salida (.o) con un compilador:

```
gcc cuda_source.o pthreads_source.o -L$CUDA_PATH/lib64 -pthread -lcudart
```

Modularidad

CUDA + OpenMP + MPI

24

- Compilación con un **único compilador**:

```
nvcc -Xcompiler -fopenmp -I$MPI_PATH/include -L$MPI_PATH/lib -lmpi mpi_source.c omp_source.c  
cuda_source.cu
```

- **Compilación separada:**

- **Compilar** cada herramienta con su compilador:

- **MPI:** `mpicc -c mpi_source.c -o mpi_source.o`
- **OpenMP:** `gcc -fopenmp -c omp_source.c -o omp_source.o`
- **CUDA:** `nvcc -c cuda_source.cu -o cuda_source.o`

- **Linkear** los archivos de salida (.o) con un compilador:

```
mpicc mpi_source.o omp_source.o cuda_source.o -L$CUDA_PATH/lib64 -lcudart -lgomp
```

Modularidad

CUDA + Pthreads + MPI

25

- Compilación con un **único compilador**:

```
nvcc -I$MPI_PATH/include -L$MPI_PATH/lib -lmpi mpi_source.c pthreads_source.c  
cuda_source.cu
```

- **Compilación separada**:

- **Compilar** cada herramienta con su compilador:

- **MPI:** `mpicc -c mpi_source.c -o mpi_source.o`
- **Pthreads:** `gcc -pthread -c pthreads_source.c -o pthreads_source.o`
- **CUDA:** `nvcc -c cuda_source.cu -o cuda_source.o`

- **Linkar** los archivos de salida (.o) con un compilador:

```
mpicc mpi_source.o pthreads_source.o cuda_source.o -L$CUDA_PATH/lib64 -lcudart
```

Modelos híbridos

CUDA + OpenMP

CUDA + Pthreads

CUDA + MPI

CUDA + OpenMP + MPI

CUDA + Pthreads + MPI

Modularidad

Distribución de carga

Distribución de carga

27

- Una arquitectura híbrida es una arquitectura heterogénea:
 - Las unidades de procesamiento son diferentes, con distintas potencias de cómputo.
- Para un problema con determinada carga de trabajo, se requiere que cada unidad de procesamiento reciba parte de la carga proporcional a su potencia de cómputo.
- Debería balancearse la carga de trabajo de manera que todas las unidades de procesamiento terminen el trabajo “casi al mismo tiempo” y ese tiempo debería ser el mínimo (el que maximice el rendimiento).
- El problema de distribución de carga balanceada es **NP-Completo**.
 - Una solución a este problema no puede automatizarse. Sin embargo, existen varias estrategias para permitir un balance de carga aceptable.

